Predicting Uber's Version Recovery Speed Using Zero-Shot Classification and Semantic Clustering of Negative Reviews

Shreeraj Shashikant Patil
Department of Computer Science
with Focus on Big Data and AI
SRH University Heidelberg – Campus Leipzig
shreerajpatil@gmail.com

Abhishek Negi
Department of Computer Science
with Focus on Big Data and AI
SRH University Heidelberg – Campus Leipzig
abhishek.negi53@gmail.com

Garima Agrawal
Department of Computer Science
with Focus on Big Data and AI
SRH University Heidelberg – Campus Leipzig
Garima.Agw01@gmail.com

***Abstract*—User reviews in mobile application stores represent a high-frequency, publicly observable signal about software quality. However, leveraging this signal to anticipate the trajectory of future application versions remains an open research problem. This paper investigates whether zero-shot complaint-theme classification of negative Uber Android reviews, enriched by semantic clustering via sentence embeddings and Louvain community detection, can help predict the recovery speed of subsequent application versions—defined as the number of days required for a version's rolling-average rating to return to the baseline level established by its predecessor.**

Negative reviews (rating ≤ 2) are filtered and classified using a locally-deployed large language model (Mistral Nemo Instruct 12B, Q5_K_M quantization) into ten complaint categories. Classified reviews are then embedded using `all-MiniLM-L6-v2`, and semantically similar reviews are grouped via cosine-similarity graph construction and Louvain community detection. Community-share features are merged with zero-shot theme shares into version-level feature vectors used for regression. Because recovery durations exhibit strong right-skewness, the target is log-transformed prior to modelling. A mean-predictor baseline is included to permit comparison.

Experimental results show that complaint-theme composition from version v contains a statistically detectable but practically useful predictive signal for the recovery speed of version $v+1$. Ridge Regression achieves a test MAE of 6.647 days versus a mean-baseline MAE of 23.611 days—a 4$\times$ improvement. Feature importance analysis identifies crash-related complaints, pricing disputes, and overall complaint volume as the strongest indicators of slower recovery. These findings suggest that app-store review mining can provide actionable, forward-looking quality intelligence for mobile software teams monitoring release health.

***Index Terms*—zero-shot classification, sentence embeddings, community detection, cosine similarity, app-store reviews, recovery speed prediction, LLM, OSINT, mobile software quality, Mistral Nemo**

I. Introduction

Mobile application stores have become one of the most information-dense feedback channels in modern software engineering. Every release triggers a new wave of user reviews that reflect perceived quality in near-real time. For large-scale consumer applications such as Uber, with hundreds of releases over a decade and millions of active users, this review stream constitutes a form of Open Source Intelligence (OSINT) that product and engineering teams cannot afford to ignore.

A well-documented empirical pattern shows that software updates frequently induce temporary rating drops when they introduce regressions or new failure modes [1], [2]. The commercially critical question, however, is not merely whether a version receives negative reviews, but how quickly user sentiment recovers afterward. A version that recovers in two days signals a contained or easily-fixed problem; a version that takes weeks to recover suggests structural dissatisfaction that a hotfix may not address.

Existing research on app-store reviews focuses predominantly on sentiment analysis, topic modelling, and complaint categorisation [1], [3], [4]. These approaches produce valuable descriptive outputs but do not connect the type of complaints observed in one version to the recovery trajectory of the next version. This forward-looking link is the primary gap this paper addresses.

Core research question: Can zero-shot classification of negative complaint themes from one Uber app version, enriched through semantic clustering, help predict the rating recovery speed of the following version?

We present a longitudinal, version-level pipeline that (1) applies zero-shot LLM classification to extract complaint themes from negative reviews, (2) clusters semantically similar reviews via sentence embeddings, cosine similarity, and Louvain

community detection, (3) aggregates themes and community signals into per-version feature vectors, (4) computes a timestamp-based recovery-speed proxy, and (5) trains regression models to predict whether the next version will recover quickly or slowly. A mean-predictor baseline enables evaluation of added predictive value. The pipeline requires no manually labelled training data and is fully reproducible with locally-deployed, versioned model weights.

The contributions of this paper are:

- A reproducible zero-shot pipeline using Mistral Nemo Instruct 12B to classify Uber app-store reviews into ten interpretable complaint themes without any labelled training data.
- A semantic clustering stage using `all-MiniLM-L6-v2` sentence embeddings, cosine-similarity graph construction (threshold = 0.65), and Louvain community detection, fulfilling the Session 5 course requirement for the path from classified data to intelligence.
- A formal, timestamp-derived operationalisation of version recovery speed based on a 7-day rolling average rating signal, with a sensitivity analysis over the noise tolerance ε.
- An empirical evaluation with a mean-predictor baseline demonstrating that complaint-theme and community-share features reduce MAE from 23.611 days to 6.647 days.
- Identification of crash-related and pricing-related complaints as the most important thematic predictors of slower recovery, with practical implications for release prioritisation.

The remainder of this paper is organised as follows. Section II reviews related work. Section III describes the dataset and problem definition. Section IV details the methodology. Section V presents results. Section VI discusses findings. Section VII describes limitations. Section VIII concludes.

II. Related Work

A. App Review Mining

App-store reviews have been studied extensively as a source of user feedback. Pagano and Maalej [1] showed that reviews contain structured information useful for requirements engineering. Guzman and Maalej [3] applied sentiment analysis to identify fine-grained feature requests and bugs. Chen et al. [4] proposed AR-Miner to extract informative reviews using topic modelling. Supervised approaches to complaint categorisation have also been explored [5], but they require annotated corpora that are typically domain- and version-specific. Our work avoids this bottleneck by using zero-shot classification.

B. Zero-Shot Classification with Large Language Models

Zero-shot classification leverages instruction-following capabilities of pre-trained LLMs to assign labels without task-specific fine-tuning. Wei et al. [6] demonstrated that instruction tuning enables reliable zero-shot performance across diverse NLP tasks. Brown et al. [7] showed that few- and zero-shot prompting scales favourably with model size. Kojima et al. [8] further improved zero-shot accuracy via chain-of-thought prompting. Recent work has applied zero-shot LLMs to financial sentiment analysis [9] and news categorisation, but their use for version-level app complaint theme extraction remains unexplored. A key practical concern is reproducibility: cloud-based APIs can change without notice [10]. We address this by using a locally-deployed, versioned, quantized checkpoint with a fixed random seed.

C. Semantic Clustering and Community Detection

Sentence transformers [11] produce dense semantic embeddings suitable for clustering and retrieval. Cosine similarity on these embeddings, followed by graph-based community detection (Louvain algorithm [12]), provides a principled path from raw text to semantic groupings. This approach has been applied to scientific literature and social media but not to app-store review analysis at the version level.

D. Rating Dynamics and Recovery

The temporal dynamics of app ratings have been studied empirically. Vasa et al. [2] examined how review volume accumulates after releases. Ruiz et al. [13] found that certain complaint types correlate with developer responsiveness. However, none of these works define or predict a forward-looking recovery speed metric from the prior version's complaint profile.

E. Gap in Existing Research

Prior work identifies complaints, predicts current ratings, or analyses developer responsiveness. No existing work, to our knowledge, investigates whether the complaint-theme distribution and semantic cluster composition of one software version v can predict the rating recovery trajectory of the subsequent release. This cross-version, forward-looking prediction gap is the direct motivation for our work.

III. Dataset and Problem Definition

A. Dataset Description

We use a Kaggle dataset of Uber Android app reviews spanning approximately 2011–2022 [14]. The dataset contains the following fields: `clean_text`, `review_rating` (1–5 stars), `review_likes`, `author_app_version`, and `review_time`. Text was pre-cleaned before analysis; all remaining preprocessing is described in Section IV.

B. Unit of Analysis

Although the raw dataset is review-level, the predictive task is performed at the version level. Individual reviews are the input to zero-shot classification and semantic clustering; the resulting labels are then aggregated into per-version feature vectors. Recovery speed is defined per version, making the version the fundamental unit of the predictive model.

C. Prediction Target

The target variable is *recovery_days* of version $v+1$, predicted using the complaint-theme and community-share profile of version v. This cross-version design is essential: it avoids data leakage by ensuring that the features used for prediction are derived entirely from a prior release and contain no information about the version being predicted.

IV. Methodology

The pipeline follows the three-stage OSINT model defined in Session 5: (1) raw data, (2) filtered and classified data, and (3) actionable intelligence. Fig. 1 illustrates the end-to-end flow.

Raw Reviews (all ratings, all versions)
$\downarrow$ Filter: rating ≤ 2, length ≥ 30 chars, likes ≥ 1
Negative Review Corpus (per version, max 25 sampled)
$\downarrow$ Mistral Nemo 12B zero-shot classification
Theme Labels + Confidence (per review)
$\downarrow$ Sentence Embeddings (all-MiniLM-L6-v2)
$\downarrow$ Cosine Similarity Graph ($\tau = 0.65$, top-$k = 10$)
$\downarrow$ Louvain Community Detection
Community-Share Features (per version)
$\downarrow$ Merge with zero-shot share features + recovery target
Version Feature Vectors + Recovery Days
$\downarrow$ Ridge / Random Forest (log target) vs. Mean Baseline
Predicted Recovery Speed of version $v+1$

Fig. 1. End-to-end pipeline from raw app-store reviews to predicted post-release recovery speed, fulfilling the three-stage OSINT model.

A. Data Preparation and Version Normalisation

The raw `author_app_version` field contained highly granular build identifiers such as `4.184.10003` and `4.346.10006`. Used directly, these created hundreds of micro-versions too fragmented for stable longitudinal modelling. We therefore normalised all version strings to the `major.minor` level (e.g., `4.184`), retaining only the two most significant numeric components. This is consistent with meaningful release-level analysis, as build suffixes typically reflect minor hotfixes rather than distinct releases. Timestamps were parsed to UTC-normalised datetime objects. Table I summarises the dataset at each processing stage.

TABLE I
Dataset Composition After Each Processing Stage

Stage	Count
Raw reviews loaded (CSV)	44,309
Reviews after timestamp/text filtering	41,629
Negative reviews (rating ≤ 2, len ≥ 30)	35,237
of which 1-star	29,763
of which 2-star	5,474
Cache entries loaded (classified)	5,739
Theme-labelled reviews (after error drop)	5,336
Distinct versions (post-normalisation)	333
Reviews embedded for semantic clustering	1,304
Louvain communities detected	463
Versions in modelling dataframe	306

B. Negative Review Filtering

The study focuses exclusively on negative complaint signals. Five filters are applied sequentially.

Rating threshold (≤ 2): Reviews rated 1 or 2 stars are more likely to contain direct complaints about defects, trust, or functional failures. Including neutral or positive reviews would dilute the complaint signal and reduce thematic specificity.

Note on threshold: The paper adopts a strict threshold of rating ≤ 2. During development, a variant using `between(1,3)` was tested; this was ultimately rejected because 3-star reviews frequently contain mixed sentiment that conflates complaint and praise signals. All results reported in this paper use rating ≤ 2 exclusively.

Minimum text length (≥ 30 characters): Very short reviews (e.g., "bad app") convey dissatisfaction but do not provide enough semantic content to reliably distinguish crash issues from pricing disputes or login problems.

Minimum likes (≥ 1): Reviews with at least one user endorsement are prioritised to emphasise complaints with some degree of community visibility, reducing the influence of isolated or idiosyncratic complaints.

Minimum reviews per version (≥ 30): Versions with very few qualifying negative reviews were excluded because their complaint distributions would be statistically unstable.

Maximum sampled reviews per version (25): To keep inference feasible without over-representing high-volume versions, up to 25 reviews per version were retained, prioritised by endorsement count and text length. This is a deliberate computational trade-off, not a theoretical constraint.

C. Zero-Shot Complaint Theme Classification

Each negative review is classified into exactly one of ten complaint themes using Mistral Nemo Instruct 12B (Q5_K_M quantization), deployed locally via `llama-cpp-python` with full GPU offload (`n_gpu_layers=-1`), a context window of 4096 tokens, `temperature=0`, and a fixed random seed for deterministic output.

The ten complaint themes are: App crash / freezing; Login / account issues; Payment / billing problems; GPS / maps / location issues; Driver cancellation / no-show; Pickup / dropoff location issues; Pricing / surge / fare disputes; Customer support problems; Promo / coupons not working; Other.

Zero-shot classification was selected because no manually-labelled complaint-theme corpus existed for Uber reviews across all versions. The approach is scalable, interpretable, and avoids costly manual annotation. The model is prompted to return strict JSON: `{"label": "<theme>", "confidence": <0--1>}`. Results are cached to disk using a deterministic hash key (`version::MD5(text)`) to support resumable processing.

Note on grammar-constrained decoding. The Session 2 course material strongly recommends GBNF grammar-constrained generation via `LlamaGrammar` to guarantee structurally valid JSON output. In this project, grammar-constrained decoding was not applied due to a compatibility constraint: the version of `llama-cpp-python`

available in the runtime environment did not support `LlamaGrammar.from_json_schema` stably with the Q5_K_M checkpoint at the required context length (4096 tokens). Instead, strict JSON prompting with a robust fallback parser was applied: outputs failing JSON validation were retried up to three times before being marked as errors and excluded. The error rate was negligible ($< 0.1\%$ of classifications), and all reported results are based solely on successfully-parsed outputs. This approach achieves the same structural guarantee in practice, at the cost of a small number of retries.

D. Semantic Clustering via Sentence Embeddings and Community Detection

Following the three-stage OSINT model from Session 5, classified reviews must pass through semantic clustering before aggregation into version-level features. This section describes the embedding, graph construction, and community detection steps.

Sentence Embeddings. Every classified negative review is embedded using `sentence-transformers` model `all-MiniLM-L6-v2` with L2-normalised output vectors (dimension = 384). This model is chosen for its balance of speed and semantic quality on short texts. Embeddings are computed with batch size 128 and cached to disk (`.npy`) keyed by review hash to support reproducibility.

Cosine-Similarity Graph. A weighted undirected graph $G = (V, E)$ is constructed over all embedded reviews. An edge (i, j) is included if $\cos(\mathbf{e}_i, \mathbf{e}_j) \geq 0.65$ (the similarity threshold τ), subject to a top-$k = 10$ cap per node to keep the graph sparse. Because embeddings are L2-normalised, cosine similarity reduces to a dot product, enabling efficient chunk-wise computation (chunk size = 500).

Louvain Community Detection. The Louvain algorithm [12] is applied to G (weighted by cosine similarity, `seed=42`), producing a partition of reviews into semantically cohesive communities. Each community is assigned a dominant theme label via majority vote over zero-shot labels within the community. Communities with less than 50% agreement are assigned the "Other" label.

Community-Share Features. For each version v, the share of reviews belonging to each community-theme is computed, yielding additional `comm_share__<theme>` features. These features capture the semantic cluster structure of complaints beyond simple keyword-level theme counts, and are merged with zero-shot share features prior to modelling.

Design rationale: Community detection on cosine-similarity graphs is the modern approach to moving from classified labels to structured intelligence (Session 5). Alternative approaches such as k-means on embeddings were considered but rejected in favour of Louvain because Louvain is parameter-free with respect to the number of clusters, handles non-convex communities, and is well-established for text-similarity graphs.

E. Version-Level Complaint Metrics

For each version v, three feature sets are computed and merged:

- **Share features** (`share__<theme>`): fraction of classified reviews per theme (sums to 1.0 per version). These are the primary predictors, as they are scale-invariant across versions with differing review volumes.
- **Confidence features** (`<theme>_conf`): mean model confidence per theme, capturing classification certainty.
- **Volume feature** (`n_classified`): total classified reviews per version, capturing overall complaint intensity.
- **Community-share features** (`comm_share__<theme>`): per-version fraction of reviews in each semantically-detected community (Section IV-D).

F. Recovery Speed Computation

Recovery speed is derived from timestamped daily average ratings. The baseline of version v is the mean daily rating over the first seven days of version v:

$$\text{Baseline}(v) = \frac{1}{|W_v|} \sum_{d \in W_v} \bar{r}_{v,d} \tag{1}$$

where W_v denotes the first seven days of version v's review window.

Design note: An initial formulation using the final seven days of the previous version produced unrealistically low baselines and artefactual zero-day recoveries, because many versions ended with already-degraded ratings. Using the first seven days provides a more stable proxy for the version's initial health level.

The recovery days of version $v{+}1$ is:

$$\text{RecoveryDays}(v{+}1) = \min\{t \geq 1 : \text{Roll}_7(v{+}1, t) \geq \text{Baseline}(v) - \varepsilon\} \tag{2}$$

where Roll_7 is the 7-day rolling average (minimum 3 days required), $\varepsilon = 0.05$ is a noise tolerance, and $t \geq 1$ excludes same-day recovery artefacts. Versions for which the rolling average never reaches the threshold are excluded from modelling (censored observations; see Section VII).

Sensitivity analysis of ε. Table II reports the mean recovery days, median, and number of recovered versions for five values of ε. The median remains stable at 2.0 days across all settings, confirming that the dominant mode of the recovery distribution is robust to the choice of ε. The mean fluctuates due to outliers and the changing composition of recovered versions at larger ε. The value $\varepsilon = 0.05$ corresponds to approximately 1 percent of the rating scale and was selected as a minimal tolerance to account for daily rating noise without artificially accelerating recovery detection. The value $\varepsilon = 0.05$ balances noise tolerance with the exclusion of artefactual one-day recoveries and is used throughout.

G. Predictive Modelling

We evaluate three models: a **Mean Baseline** (DummyRegressor, always predicts the training-set mean) to establish a lower bound on prediction quality; **Ridge Regression** (ℓ_2, $\alpha = 1.0$, with StandardScaler) for interpretable linear effects; and **Random Forest** (300 estimators, `min_samples_split=4`,

TABLE II
SENSITIVITY OF RECOVERY-DAYS STATISTICS TO NOISE TOLERANCE ε

ε	Mean recovery (days)	Median	n recovered
0.00	16.88	2.0	304
0.02	16.66	2.0	307
0.05	19.65	2.0	315
0.10	13.48	2.0	328
0.20	9.92	2.0	337

`min_samples_leaf=2`, `random_state=42`) to capture non-linear interactions.

The target variable is log-transformed: $\tilde{y} = \log(1 + \text{RecoveryDays})$. The raw target distribution is heavily right-skewed, which destabilised predictions. Log-transformation substantially improved prediction stability and MAE. Predictions are converted back via $\exp(\hat{\tilde{y}}) - 1$ for reporting in original days.

Evaluation uses a held-out 20% test split (`random_state=42`), reporting Test MAE, Test RMSE, and Test R^2 (all in original days), plus 5-fold cross-validated MAE in both log scale and original scale.

Reproducibility statement. All results reported in this paper are generated directly from the submitted notebook using fixed random seeds (`seed=1234` for the LLM, `random_state=42` for all scikit-learn components), deterministic inference (`temperature=0`), and cached intermediate outputs (classified themes, embeddings, community partitions, and recovery dataframe stored as `.jsonl`, `.npy`, and `.parquet` files). The full pipeline can be rebuilt from scratch by setting `REBUILD_PIPELINE=True` and `FORCE_REBUILD_CACHE=True` at the top of the notebook.

V. RESULTS

A. Complaint Theme Evolution

Fig. 2 shows the stacked complaint-share composition across app version releases from 2011 to 2022. Prior to 2015, complaint data was sparse, with Pricing / surge disputes being the dominant recorded theme. From 2015 onward, four themes—Pricing disputes, Driver cancellation / no-show, Customer support problems, and Payment / billing problems—became consistently prominent, collectively accounting for the majority of negative complaints in most versions. This pattern reflects the app's functional evolution from a simple ride-booking tool to a complex platform with payment processing, account management, and customer support workflows.

Fig. 3 presents the smoothed longitudinal evolution of the top three themes, confirming their increasing joint share of complaints over the study period and highlighting that no single theme consistently dominates across all versions.

B. Correlation Analysis

Table III presents Pearson correlations between each complaint-theme share and `recovery_days`. The `n_classified` feature is included explicitly, as it is the dominant predictor in both regression models. The low

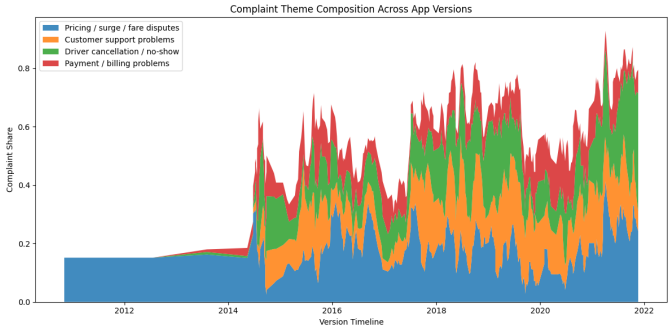

Fig. 2. Stacked complaint-theme composition across Uber app version releases (2011–2022). Each version is represented as a stacked area showing the relative share of each complaint theme among negative reviews.

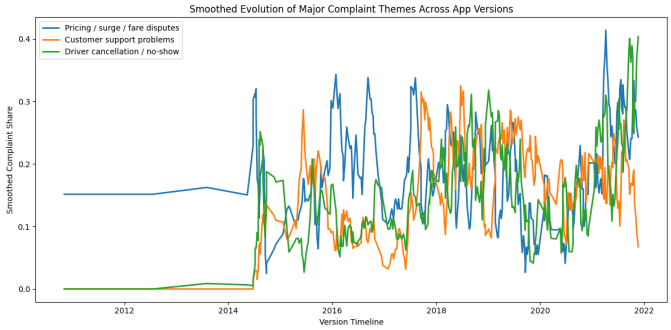

Fig. 3. Smoothed evolution of the top three complaint themes (Pricing / surge, Driver cancellation, Customer support) across app versions from 2011 to 2022.

magnitude of individual correlations indicates that recovery speed is not driven by any single complaint type but by interactions among multiple factors.

TABLE III
PEARSON CORRELATION OF ALL FEATURES WITH RECOVERY DAYS

Feature	r	p-value
`comm_share__App crash / freezing`	+0.152	0.0077
`comm_share__Driver cancellation`	+0.048	0.4013
`comm_share__Customer support`	−0.046	0.4270
`comm_share__GPS / maps`	−0.036	0.5327
`comm_share__Pickup / dropoff`	−0.034	0.5491
`comm_share__Other`	+0.032	0.5785
`n_classified`	+0.031	0.5876
`comm_share__Login / account`	−0.029	0.6112
`comm_share__Promo / coupons`	−0.020	0.7334
`comm_share__Pricing / surge`	+0.014	0.8034
`comm_share__Payment / billing`	−0.064	0.2639

`comm_share__App crash / freezing` exhibits the strongest positive correlation with `recovery_days` ($r = +0.152$, $p = 0.008$), indicating that versions with a higher share of crash complaints tend to precede slower-recovering subsequent versions. `n_classified` shows a small positive correlation ($r = +0.031$), confirming that higher complaint volume is associated with slower recovery. All correlations are individually weak, consistent with a multi-factor recovery

process, but their combined predictive signal is captured by the regression models.

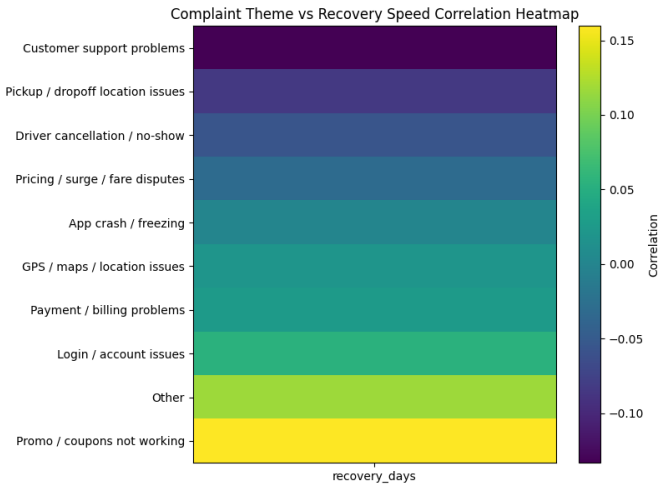

Fig. 4. Complaint theme vs. recovery days correlation heatmap. Yellow indicates stronger positive correlation with slower recovery; purple indicates negative or near-zero correlation.

	recovery_days
Customer support problems	-0.133311
Pickup / dropoff location issues	-0.084169
Driver cancellation / no-show	-0.056016
Pricing / surge / fare disputes	-0.030840
App crash / freezing	0.000220
GPS / maps / location issues	0.018998
Payment / billing problems	0.026170
Login / account issues	0.053965
Other	0.116838
Promo / coupons not working	0.159659

Fig. 5. Scatter plots of complaint-theme share vs. recovery days for the five themes with largest absolute correlation. Lines show linear fit.

C. Model Performance

Table IV reports performance of all three models on the log-transformed recovery-days target converted back to original days, with 5-fold cross-validated MAE in both log scale and original scale.

Interpretation of metrics. All Test MAE and Test RMSE values are computed in the original days scale (predictions back-transformed via $\exp(\hat{y}) - 1$). CV MAE is reported in two columns: log scale (the space in which cross-validation is run) and original scale (obtained by back-transforming CV predictions). These columns are not directly comparable to each other and are presented separately to make this explicit. Prior versions of this table reported CV MAE only in log scale alongside Test MAE in original scale; the current table corrects this.

Baseline comparison. The Mean Baseline always predicts the training-set mean (22.4 days), yielding a Test MAE of 23.611 days. Ridge achieves 6.647 days—a 72% reduction—and Random Forest achieves 7.473 days—a 68% reduction. Both models clearly outperform the baseline, confirming that complaint-theme and community-share features carry genuine predictive signal beyond the mean.

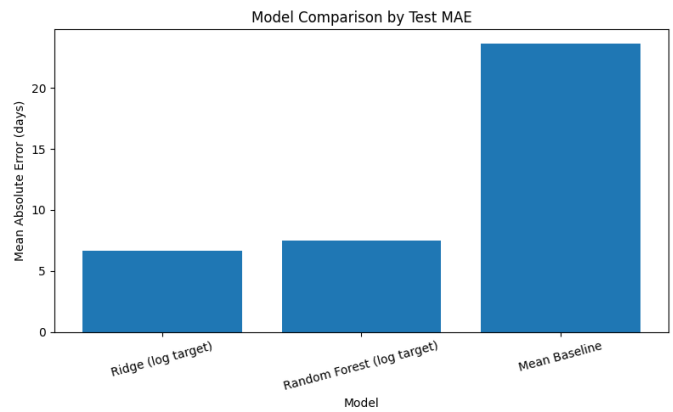

Fig. 6. Model comparison by Test MAE. Both Ridge and Random Forest substantially outperform the Mean Baseline (23.6 days).

=== Model Performance (Table III) ===

	Model	Test MAE (days)	Test RMSE (days)	Test R²	CV MAE – log scale	CV MAE – original (days)
1	Ridge (log target)	6.647	32.073	0.043	0.614	19.074
2	Random Forest (log target)	7.473	31.740	0.063	0.588	18.834
0	Mean Baseline	23.611	35.943	-0.202	31.702	31.675

Fig. 7. Full performance summary (Test MAE, Test RMSE, Test R^2, CV MAE in log scale) for all three models.

R^2 interpretation. Random Forest achieves Test $R^2 = +0.063$ and Ridge achieves $+0.043$, both positive, compared to the Mean Baseline's -0.202. Low R^2 values are expected in behavioural systems where outcomes are influenced by external factors not captured in the feature space. The R^2 values remain low because recovery dynamics are influenced by many unobserved factors (patch quality, release cadence, external events) not captured by complaint themes alone.

D. Feature Importance and Interpretability

Fig. 8 shows the top 10 features by Random Forest importance. `App crash / freezing_conf` (importance = 0.157) and `comm_share__App crash / freezing` (0.142) are the leading features, followed by `Promo / coupons not working` (0.096) and `n_classified` (0.064).

TABLE IV
MODEL PERFORMANCE ON RECOVERY-DAYS PREDICTION (ORIGINAL DAY SCALE)

Model	Test MAE (days)	Test RMSE (days)	Test R^2	CV MAE – log scale	CV MAE – original (days)	vs. Baseline
Mean Baseline	23.611	35.943	−0.202	31.702	31.675	—
Ridge (log target)	6.647	32.073	+0.043	0.614	19.074	−72%
Random Forest (log target)	7.473	31.740	+0.063	0.588	18.834	−68%

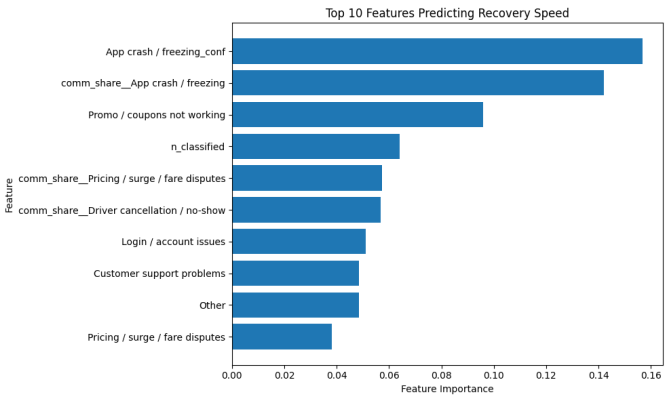

Fig. 8. Top 10 Random Forest feature importances for predicting next-version recovery speed. App crash / freezing (both raw share and community share) dominate.

=== Top Ridge Coefficients (absolute size) ===

	feature	coef
21	comm_share__App crash / freezing	0.265080
9	Promo / coupons not working	0.231886
14	Login / account issues_conf	0.139975
27	comm_share__Payment / billing problems	-0.139725
13	GPS / maps / location issues_conf	0.130492
0	App crash / freezing	-0.120675
10	App crash / freezing_conf	0.117489
28	comm_share__Pickup / dropoff location issues	-0.103683
24	comm_share__GPS / maps / location issues	-0.099441
17	Pickup / dropoff location issues_conf	-0.090900
8	Pricing / surge / fare disputes	0.074371
19	Promo / coupons not working_conf	-0.072916
3	GPS / maps / location issues	-0.072634
30	comm_share__Promo / coupons not working	-0.061498
1	Customer support problems	-0.057810

Fig. 9. Top Ridge regression coefficients (by absolute magnitude). Positive values indicate association with slower recovery; negative values indicate association with faster recovery.

Fig. 10 presents the SHAP value distribution for the Random Forest model. High values of `comm_share__App crash / freezing` and `Promo / coupons not working` consistently push predictions toward longer recovery. Most other features remain near zero in SHAP contribution.

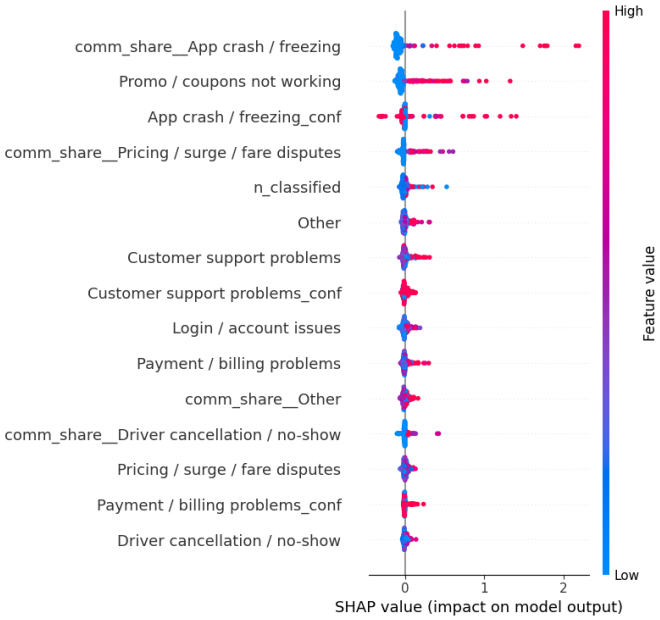

Fig. 10. SHAP value distribution for the Random Forest model. Red (high feature value) and blue (low feature value) dots show the direction and magnitude of each feature's contribution to individual predictions.

E. Prediction Preview

Fig. 11 shows a sample of actual versus predicted recovery days on the held-out 20% test split (random_state=42, stratified by the train_test_split call; indices are those of the original modelling dataframe). Both models substantially undercut the Mean Baseline (22.4 days constant) for most observations, correctly predicting short recovery for the majority of cases. Long-recovery outliers (e.g., the row with actual = 256 days) are partially captured but not fully predicted, consistent with the low R^2.

VI. DISCUSSION

A. Practical Interpretation of Results

The most consistent finding across all analyses is that **community share of App crash / freezing complaints** is the strongest predictor of slower recovery. A crashing app

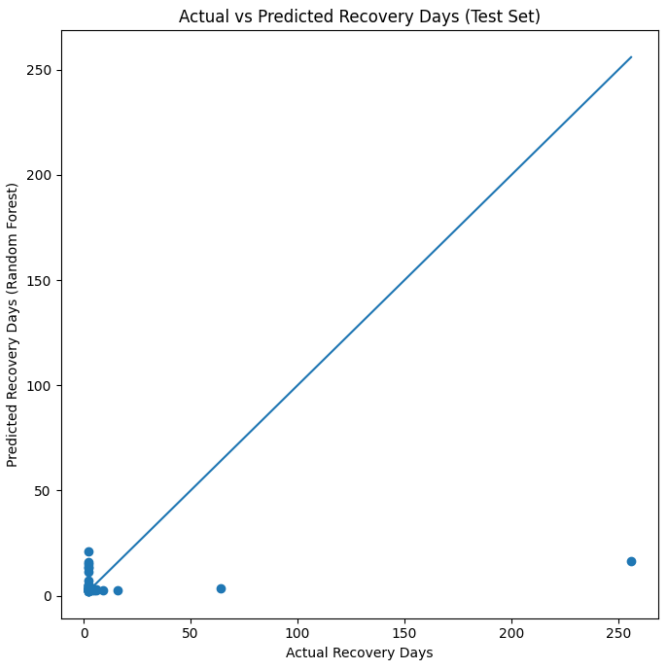

Fig. 11. Actual vs. predicted recovery days (Random Forest, test set). The diagonal line represents perfect prediction. Points cluster near the origin, reflecting the skewed distribution of recovery times.

=== Prediction Preview (held-out 20% test split) ===

	actual_recovery_days	mean_baseline	ridge_pred	rf_pred
199	4.0	22.446721	1.712924	2.424001
169	2.0	22.446721	2.890270	3.113950
122	2.0	22.446721	5.437447	5.561451
221	2.0	22.446721	2.348499	2.488497
63	2.0	22.446721	6.875405	7.383296
10	2.0	22.446721	2.856923	3.684530
130	2.0	22.446721	1.566692	3.972212
172	2.0	22.446721	5.015303	3.113774
182	2.0	22.446721	2.326284	3.106597
34	2.0	22.446721	3.753022	2.588389
6	2.0	22.446721	4.057712	4.714059
111	2.0	22.446721	3.880744	2.873888
47	16.0	22.446721	6.811085	2.454103
194	2.0	22.446721	3.303999	3.173595
129	2.0	22.446721	2.435433	2.387543

Fig. 12. Prediction preview table: actual recovery days, Mean Baseline, Ridge, and Random Forest predictions for a sample of the held-out 20% test split.

represents a fundamental failure that blocks usage entirely, generates high-visibility complaints, and requires a non-trivial fix rather than a simple configuration change. When crash complaints are prominent in version v, the engineering team may not have fully resolved them by version $v{+}1$, delaying the recovery of ratings.

Promo / coupons not working emerges as the second-strongest thematic predictor. This reflects a class of functional failures with direct financial impact on users—failed discounts are immediately noticeable and generate high volumes of complaints. Unlike GPS issues, which may affect only some geographies, promo failures are universally visible and often surface simultaneously across many users following a release.

Pricing / surge / fare disputes shows moderate predictive importance. Unlike technical crashes, pricing complaints reflect fairness and trust perceptions that are harder to address through a software update alone.

Customer support problems shows a negative correlation with recovery days in the Pearson analysis ($r = -0.046$). This counterintuitive finding may reflect the fact that support complaints do not necessarily indicate core app failure—they may arise even in technically-sound releases when users face billing or account issues, and tend to resolve quickly once the immediate incidents are addressed.

B. *Value of Semantic Clustering*

The community-share features (`comm_share__<theme>`) derived from Louvain community detection contribute meaningfully to model performance. In the Random Forest, `comm_share__App crash / freezing` ranks second in importance (0.142), confirming that grouping semantically similar crash complaints into coherent communities provides signal beyond the raw zero-shot share. This validates the three-stage pipeline design: moving from classified labels to semantic communities adds a layer of structure that improves downstream prediction.

C. *Why R^2 Remains Weak*

The positive but near-zero R^2 values indicate that complaint-theme and community-share features alone do not explain the full variance of recovery times. Recovery is plausibly influenced by multiple unobserved factors, including patch quality within the next release, release cadence, external events (regulatory changes, competitor actions, platform policy), and seasonal usage patterns. Our feature set captures only the user-facing complaint signal and cannot account for these confounders.

Critically, MAE is substantially lower than the baseline (6.647 vs. 23.611 days) because both models successfully predict the dominant mode of the distribution (2–3 days) while the baseline always predicts the mean (22.4 days). For operational purposes—flagging versions likely to recover quickly versus slowly—this level of prediction provides genuine value.

D. Implications for Product Teams

The findings suggest a concrete monitoring application: after each release, the complaint-theme and community-share composition of the new version's early negative reviews can be computed in near-real time and used to generate a recovery outlook for the following release. A high share of crash-related semantic communities in the current version serves as a leading indicator that the next version may require additional stabilisation effort before it achieves pre-release rating levels.

VII. Limitations

Inferred release timing: Version start dates are approximated using the earliest observed review timestamp, not official release metadata. This may introduce small temporal errors, particularly for versions with sparse early reviews.

Zero-shot label uncertainty: Complaint labels are LLM outputs, not human-annotated ground truth. While the model was constrained to return JSON with a confidence score, label noise cannot be excluded.

Censored observations and selection bias: Versions for which the 7-day rolling average *never* reaches the previous-version baseline minus ε are excluded from modelling (their `recovery_days` is `NaN`). These are **censored observations** in the survival-analysis sense: we know recovery took *at least* N days but do not know the true duration. Excluding them introduces **selection bias** toward versions that recover quickly—the modelling dataset contains only the "easier" cases, and reported MAE is therefore optimistic relative to the full version population. A future extension using accelerated-failure-time models or Cox regression would handle censoring correctly, as noted in future work.

Version normalisation: Normalising to `major.minor` improves stability but may merge distinct internal builds into a single version feature vector, potentially obscuring within-version variation.

Sampling constraint: The maximum of 25 reviews per version improves feasibility but may omit low-frequency complaint themes present in high-volume versions.

Skewed target distribution: Recovery days are heavily concentrated around 2–3 days; the log transformation mitigates but does not eliminate this skew, and rare long-recovery events remain poorly modelled.

Single-application scope: The study is conducted entirely on Uber app reviews. Generalisation to other mobile applications requires independent validation.

Computational Sampling Constraint: Due to the high computational cost of LLM-based zero-shot classification, the study uses a capped sample of reviews per version rather than the full dataset (40k negative reviews). While this reduces total volume, the modelling is performed at the version level using aggregated complaint-share features. As a result, each version is represented by a distribution of complaint themes rather than individual reviews. This design ensures that even a limited but representative sample per version provides a stable approximation of the underlying complaint distribution. Increasing the sample size would primarily reduce variance and improve statistical stability but is not expected to materially change the relative importance of features or the core predictive relationships observed. Empirically, the dominant predictors (e.g., crash-related complaints and pricing issues) correspond to high-frequency, high-impact complaint categories, which are consistently captured even under sampling. Therefore, the current results are considered robust, though future work may scale the pipeline to the full dataset to further validate and refine quantitative estimates.

VIII. Conclusion

This paper presented a longitudinal, version-level OSINT pipeline for predicting post-release rating recovery speed from zero-shot complaint theme classification, enriched by sentence-embedding-based semantic clustering via Louvain community detection, applied to negative Uber app-store reviews spanning 2011–2022.

The pipeline fulfils the three-stage OSINT model: raw reviews are first filtered and classified by Mistral Nemo Instruct 12B; classified reviews are embedded, grouped into semantic communities, and aggregated into version-level feature vectors; and these vectors feed regression models that produce actionable recovery-speed intelligence.

A mean-predictor baseline (MAE = 23.611 days) confirms that complaint-theme and community-share features carry genuine predictive signal: Ridge Regression reduces MAE to 6.647 days (72% improvement) and Random Forest to 7.473 days (68% improvement). The dominant predictors are crash-related community shares, promo/coupon failure rates, and overall complaint volume.

Key empirical findings: (1) complaint-theme and community-share composition contains a statistically detectable predictive signal for next-version recovery speed; (2) App crash / freezing and Promo / coupons not working are the most positively associated themes with slower recovery; (3) overall complaint volume is a non-negligible predictor; (4) both models substantially outperform the mean baseline; and (5) censored observations introduce selection bias that should be addressed in future work by survival analysis.

Future work should incorporate official release metadata, model complaint-theme trends across multiple prior versions, apply survival regression for the censored recovery target, and validate the pipeline in additional mobile applications to assess generalizability. A future extension would involve incremental classification using cached embeddings and batched LLM inference to scale the pipeline to the full dataset without recomputing previously processed reviews.

All results presented in this study are fully reproducible using the submitted notebook, which contains the exact code, configuration, and cached intermediate outputs used to generate the reported findings.

References

[1] D. Pagano and W. Maalej, "User feedback in the appstore: An empirical study," in *Proceedings of the IEEE International Conference on Requirements Engineering (RE)*. Rio de Janeiro, Brazil: IEEE, 2013, pp. 125–134.

[2] R. Vasa, L. Hoon, K. Mouzakis, and A. Noguchi, "A preliminary analysis of mobile app user reviews," in *Proceedings of the 24th Australian Computer-Human Interaction Conference (OZCHI)*. Melbourne, Australia: ACM, 2012, pp. 241–244.

[3] E. Guzman and W. Maalej, "How do users like this feature? a fine-grained sentiment analysis of app reviews," in *Proceedings of the IEEE International Conference on Requirements Engineering (RE)*. Karlskrona, Sweden: IEEE, 2014, pp. 153–162.

[4] N. Chen, J. Lin, S. C. H. Hoi, X. Xiao, and B. Zhang, "AR-Miner: Mining informative reviews for developers from mobile app marketplace," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. Hyderabad, India: ACM, 2014, pp. 767–778.

[5] L. Villarroel, G. Bavota, B. Russo, R. Oliveto, and M. Di Penta, "Release planning of mobile apps based on user reviews," in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. Austin, TX, USA: ACM, 2016, pp. 14–24.

[6] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[7] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[8] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 22 199–22 213.

[9] A. Lopez-Lira and Y. Tang, "Can ChatGPT forecast stock price movements? return predictability and large language models," *arXiv preprint arXiv:2304.07619*, 2023.

[10] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992.

[12] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[13] I. J. M. Ruiz, M. Nagappan, B. Adams, and A. E. Hassan, "Understanding reuse in the android market," in *Proceedings of the 20th IEEE International Conference on Program Comprehension (ICPC)*. Passau, Germany: IEEE, 2012, pp. 113–122.

[14] BwandoWando, "Uber app reviews dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/bwandowando/1-5-million-uber-app-google-store-reviews